Intro Design Example

Lab 00

Designer: Scott Tippens
Class: CpE 3020
Term: Fall 2025
Date: 2025-08-24

Design Description

The Intro design demonstrates basic VHDL design principles and documentation expectations. Your reports should follow the basic format presented here for all your lab documentation and design.

The Intro component counts the number of active inputs among three input switches and then drives an led bar display with the results. The count ranges from 0 to 3 so only 3 leds will be necessary on the output.

Component

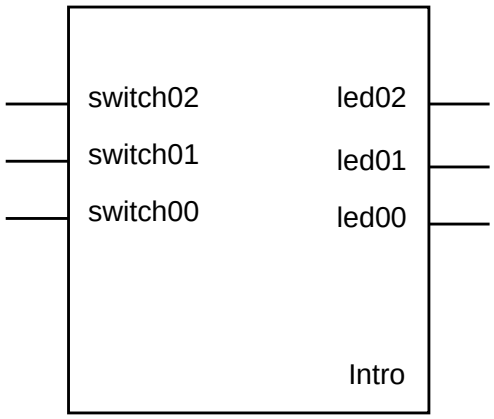

Figure 1: Intro Component Diagram

Design

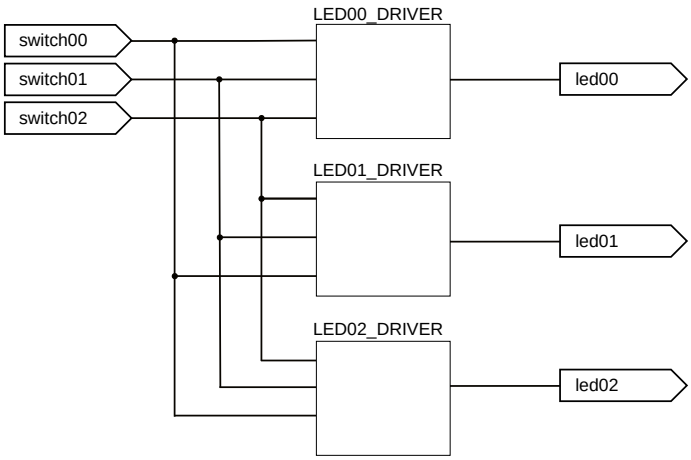

Figure 2: Intro Design Diagram

Intro Design Code

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

--*****************************************************************************
--*
--* Name: Intro
--* Designer: Scott Tippens
--*
--*     This component counts the number of active inputs (3 total).
--*     It then drives a bar LED display with the resultant count. The count
--*     will be represented with LEDs as follows: ('*' = led on; '-' = led off)
--*
--*         count = 0 --> ---
--*         count = 1 --> *--
--*         count = 2 --> **-
--*         count = 3 --> ***
--*
--*     where the order of the leds above is:
--*         count = x --> (led00) (led01) (led02)
--*
--*****************************************************************************
entity Intro is
    port (
        switch00: in std_logic;
        switch01: in std_logic;
        switch02: in std_logic;

        led00: out std_logic;
        led01: out std_logic;
        led02: out std_logic);
end entity Intro;


architecture Intro_ARCH of Intro is
begin

    LED00_DRIVER: led00 <= switch00 or switch01 or switch02;

    LED01_DRIVER: led01 <= (switch00 and switch01)
                           or (switch00 and switch02)
                           or (switch01 and switch02);

    LED02_DRIVER: led02 <= switch00 and switch01 and switch02;

end architecture Intro_ARCH;
```

Test Bench

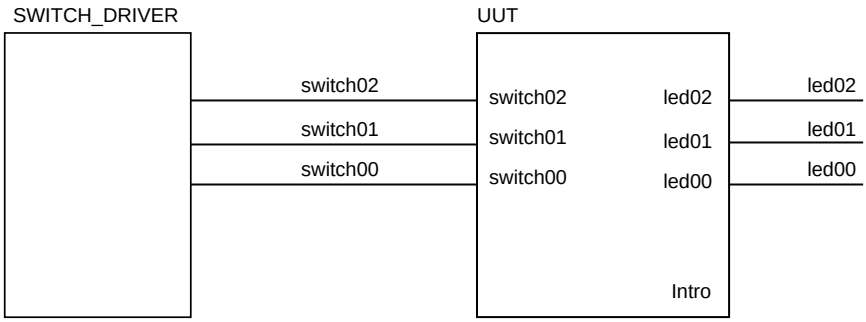

Figure 3: Intro Test Bench Diagram

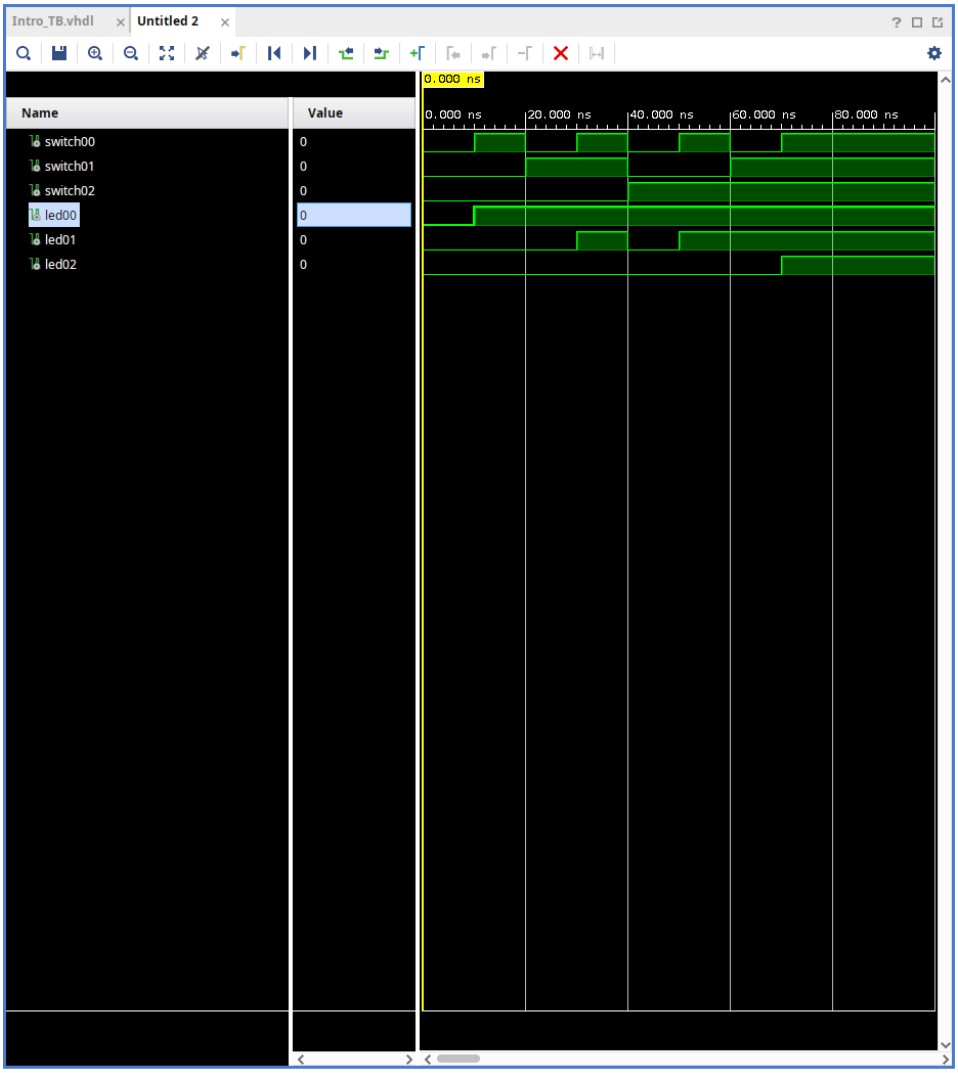

Figure 4: Intro Test Bench Waveforms

Intro Test Bench Code

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

--*****************************************************************************
--*
--* Name: Intro_TB
--* Designer: Scott Tippens
--*
--*     Test bench file for the Intro component. This test bench applies all
--*     possible signal combinations to the switch inputs using dataflow
--*     statements.
--*
--*     STUDENT NOTE: It is customary to define all signals attached to the UUT
--*     (Unit Under Test) component with the same name as the port signals of
--*     the component. This makes it easier to debug since the chance for name
--*     confusion is reduced.
--*
--*     STUDENT NOTE 2: Notice how the test bench has no ports. The test bench
--*     represents an issolated design where you define signal stimuli
--*     internally and analyze the output signals internally. No external
--*     interface is necesary.
--*
--*     STUDENT NOTE 3: You bring your design into the test bench by importing
--*     it as a component.
--*
--*****************************************************************************
entity Intro_TB is
end entity Intro_TB;


architecture Intro_TB_ARCH of Intro_TB is

    --unit-under-test-------------------------------------COMPONENT-DECLARATION
    component Intro
        port (
            switch00: in std_logic;
            switch01: in std_logic;
            switch02: in std_logic;
            led00:    out std_logic;
            led01:    out std_logic;
            led02:    out std_logic);
    end component;

    --uut-signals-------------------------------------------------------SIGNALS
    signal switch00: std_logic;
    signal switch01: std_logic;
    signal switch02: std_logic;
    signal led00:    std_logic;
    signal led01:    std_logic;
    signal led02:    std_logic;
```

```
begin

    --Unit-Under-Test---------------------------------------COMPONENT-INSTANCE
    UUT: Intro port map(
            switch00 => switch00,
            switch01 => switch01,
            switch02 => switch02,
            led00    => led00,
            led01    => led01,
            led02    => led02);


    --switch-driver-------------------------------------------------------PROCESS
    SWITCH_DRIVER: process
    begin
        switch02 <= '0';  switch01 <= '0';  switch00 <= '0';
        wait for 10 ns;

        switch02 <= '0';  switch01 <= '0';  switch00 <= '1';
        wait for 10 ns;

        switch02 <= '0';  switch01 <= '1';  switch00 <= '0';
        wait for 10 ns;

        switch02 <= '0';  switch01 <= '1';  switch00 <= '1';
        wait for 10 ns;

        switch02 <= '1';  switch01 <= '0';  switch00 <= '0';
        wait for 10 ns;

        switch02 <= '1';  switch01 <= '0';  switch00 <= '1';
        wait for 10 ns;

        switch02 <= '1';  switch01 <= '1';  switch00 <= '0';
        wait for 10 ns;

        switch02 <= '1';  switch01 <= '1';  switch00 <= '1';
        wait;  --suspend process forever
    end process;


end architecture Intro_TB_ARCH;
```

Basys3 Wrapper

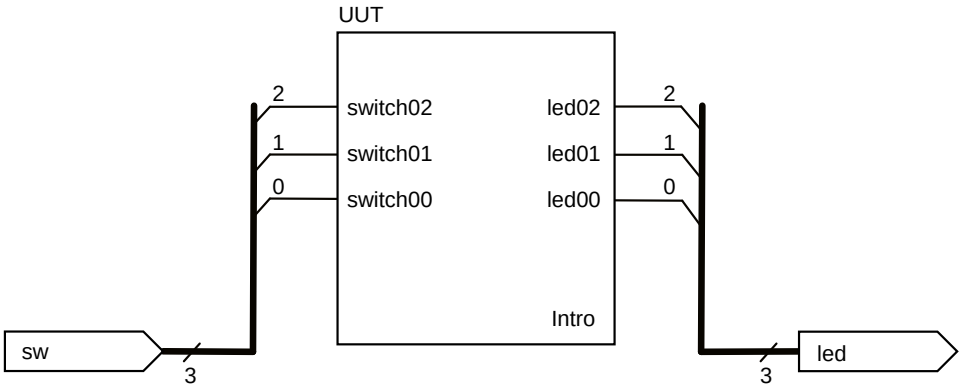

Figure 5: Intro Wrapper for Basys3

Intro Wrapper for Basys3

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

--*****************************************************************************
--*
--* Name: Intro_BASYS3
--* Designer: Scott Tippens
--*
--*     wrapper file for the Intro component. This wrapper is targeting the
--*     Basys3 board from Digilent. The configuration file for the Basys3 was
--*     retrieved from the digilent GitHub account.
--*
--*     The lower 3 slide switches are being used as the switch inputs and the
--*     lower 3 LEDs are being used for the bar display with LED0 being the
--*     least significant bit.
--*
--*     STUDENT NOTE: We are giving you a peak at using buses for interfacing
--*     externally. The configuration file provided by Digilent for the Basys3
--*     configures the slide switches and LEDs as bus inputs and outputs
--*     respectively. It is best to preserve the configuration file provided
--*     by the manufactures since updates can be made in the future that you
--*     don't want to break you design.
--*
--*     STUDENT NOTE 2: Wrapper files like this should always be used to map
--*     your components to physical hardware. Components should be designed as
--*     general as possible so they can be reused in different projects or
--*     when design needs inevitably change.
--*
--*****************************************************************************
entity Intro_BASYS3 is
    port (
        sw:   in  std_logic_vector(2 downto 0);
        led:  out std_logic_vector(2 downto 0));
end entity Intro_BASYS3;


architecture Intro_BASYS3_ARCH of Intro_BASYS3 is

    --unit-under-test-------------------------------------COMPONENT-DECLARATION
    component Intro
        port (
            switch00: in std_logic;
            switch01: in std_logic;
            switch02: in std_logic;
            led00:    out std_logic;
            led01:    out std_logic;
            led02:    out std_logic);
    end component;

begin
```

```
    --Unit-Under-Test----------------------------------------COMPONENT-INSTANCE
    UUT: Intro port map(
            switch00 => sw(0),
            switch01 => sw(1),
            switch02 => sw(2),
            led00    => led(0),
            led01    => led(1),
            led02    => led(2));


end architecture Intro_BASYS3_ARCH;
```